《深度学习》课程大作业

CIFAR-10 图像分类中的 CNN 结构对比与低能见度鲁棒性分析

——基于 ResNet-20 的深度可分离卷积、SE 注意力与归一化消融实验

姓　　名：孙蜀齐

学　　号：2421010213

专　　业：人工智能

指导教师：史朋飞

2026 年 11 月

摘　要

卷积神经网络（CNN）在图像分类任务上已经相当成熟，但在边缘设备部署和低能见度场景中，“精度—参数量—计算量—实际延迟”之间往往并不存在简单的线性关系。本报告以 CIFAR-10 为数据集，以 CIFAR 风格 ResNet-20 为基线，在完全一致的训练配方下比较了五种结构改动：深度可分离卷积、SE 通道注意力、GELU 激活、GroupNorm 归一化，以及深度可分离卷积与 SE 的组合。六个模型各训练 60 个 epoch，并在同一测试集上评估。

实验结果表明：基线 R20 的测试准确率为 **88.54%**；表现最好的变体是 **R20-GELU（89.34%）**。深度可分离卷积把参数量从 272,474 降到 40,522（**−85.13%**），MACs 从 40.81M 降到 6.65M（**−83.69%**）；但单张 CPU 延迟由 1.95 ms 升至 3.08 ms（**+57.4%**），说明在 CPU 上 FLOPs 的下降并不自动等于延迟的下降。SE 与 GELU 的精度变化分别为 +0.53 个百分点 和 +0.80 个百分点；GroupNorm 相对 BatchNorm 的变化为 -2.78 个百分点。

在 5 类合成退化（雾、低照度、运动模糊、高斯噪声、JPEG 压缩）上，基线平均准确率降至 57.48%，R20-DS-SE 为 54.11%；对 R20-DS-SE 做退化数据微调后，退化平均准确率由 54.11% 变为 65.70%（+11.59 个百分点），干净测试准确率由 86.45% 变为 85.65%（-0.80 个百分点）。最后，本文用 Grad-CAM 对比了干净与雾天图像上的关注区域，并实现了一个可上传图片、选择模型与退化强度的 Flask 演示系统。所有代码、日志、指标 JSON 与报告数字均由同一仓库产出，可通过固定随机种子复现。

关键词：卷积神经网络；ResNet；深度可分离卷积；SE 注意力；鲁棒性；CIFAR-10

目　录

摘要.....2

第 1 章　绪论.....4

第 2 章　数据集与实验设置.....6

第 3 章　模型设计.....9

第 4 章　训练结果与结构分析.....13

第 5 章　低能见度鲁棒性与场景应用.....19

第 6 章　结论与展望.....26

参考文献.....28

附录 A 代码结构.....29

附录 B 复现命令.....29

附录 C 结果文件说明.....30

页码与正文实际分页一致，由构建脚本从导出的 PDF 中回填；若在 Word 中修改内容导致分页变化，请以正文页码为准。

第 1 章　绪论

1.1 研究背景

图像分类是计算机视觉的基础任务，也是理解卷积神经网络内部组件的常用载体。从 AlexNet 到 ResNet，网络深度的增加带来了精度提升，但也带来了参数量和计算量的增长。在手机、嵌入式设备和边缘服务器上，模型往往需要在有限算力、有限内存和实时性约束下运行，因此“更少的参数与 FLOPs”成为模型设计的核心目标之一。

与此同时，真实部署环境并不总是清晰的实验室图像。雾、低照度、运动模糊、传感器噪声和 JPEG 压缩都会改变输入分布，使在干净数据上训练的模型出现明显精度下降。对道路场景而言，低能见度条件下的稳定识别具有直接的应用价值，但真实雾天数据的采集与标注成本很高。因此，在公开数据集上构造可控退化，是课程作业条件下研究鲁棒性的可行方案。

1.2 课程任务与教材实验的区别

本作业要求实现一个与教材实验三不同的经典 CNN，在开源数据集上训练到基准准确率，并完成结构修改、实验分析与场景应用。教材中与实验三对应的应用型章节以图像去雾、超分辨率重建和目标检测等流程为主线；本报告不重复这些流程，而是选择 CIFAR-10 图像分类作为任务，把重点放在 CNN 组件的受控消融、CPU 部署成本与退化鲁棒性上。两者在任务形式、数据规模和评价指标上均不同，满足“必须区别于教材实验三”的要求。

具体而言，本作业需要覆盖以下考察点：

- **深度可分离卷积**：比较参数量、计算量与精度，而不只报告一个准确率；
- **通道注意力**：实现 SE 模块并分析其收益与开销；
- **激活函数与归一化**：比较 GELU 与 ReLU、GroupNorm 与 BatchNorm；
- **模型设计与调优**：在统一配方下训练多个变体，保证比较公平；
- **场景适配**：构造低能见度退化，评估鲁棒性并做退化微调；
- **工程交付**：提供可运行代码、训练日志、复现说明与交互演示。

1.3 本文主要工作

本文的主要工作可以概括为四点。第一，实现了一个 CIFAR 风格 ResNet-20 基线，并在其基础上通过配置开关生成五个结构变体，六个模型共享同一数据划分、增强策略、优化器和学习率调度。第二，使用前向 hook 统计每个模型的参数量与 MACs，并在同一台 CPU 上测量 batch=1 与 batch=128 的推理延迟，给出“精度—成本”的多维权衡。第三，设计雾、低照度、运动模糊、高斯噪声和 JPEG 压缩五类退化，在 5 个强度上评估全部模型，并对轻量组合模型做退化数据微调。第四，使用 Grad-CAM 观察模型在干净与雾天图像上的关注区域，并提供一个可交互的 Flask 演示。

需要强调的是，本文的目标不是追求 CIFAR-10 上的最高精度。ResNet-20 是一个刻意保持较小的经典模型，在 CPU 上可以在可接受的时间内完成多次训练；这使得“控制变量、重复六个模型”成为可能。报告中的所有数字均来自 `results/metrics/` 下的 JSON 文件，并由 `scripts/build_report.py` 自动写入，避免手工抄录带来的不一致。

1.4 报告结构

第 2 章介绍数据集、划分、预处理与评价指标；第 3 章说明基线残差块及四种结构改动的设计；第 4 章给出训练结果，并从参数量、MACs、延迟、逐类精度和混淆矩阵等角度分析各组件的影响；第 5 章讨论低能见度鲁棒性、退化微调、Grad-CAM 与演示系统；第 6 章总结结论、局限与后续改进方向。

第 2 章　数据集与实验设置

2.1 CIFAR-10 数据集

CIFAR-10 由 Krizhevsky 等人整理，包含 10 个类别、共 60000 张 32×32 彩色图像，其中训练集 50000 张、测试集 10000 张，各类别数量基本均衡。10 个类别为：飞机、汽车、鸟、猫、鹿、狗、蛙、马、船、卡车。图像分辨率低、类内差异大、背景复杂，适合在有限算力下研究网络结构，而不适合直接评估高分辨率真实场景。

项目	取值	说明
图像尺寸	32 × 32 × 3	RGB，uint8 存储
类别数	10	各类 6000 张（训练 5000 / 测试 1000）
原始训练集	50000	按类别分层划分
训练 / 验证	45000 / 5000	每类 500 张验证，seed=42
测试集	10000	只在最终评估使用
归一化	CIFAR-10 均值 / 标准差	逐通道标准化

表 2-1 CIFAR-10 数据划分与预处理

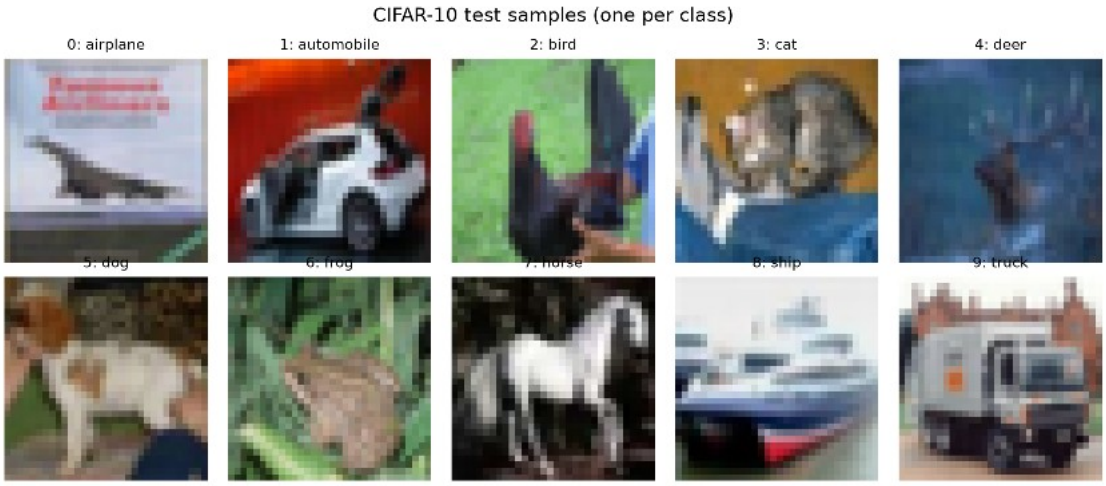


图 2-1 CIFAR-10 测试集每类一个样例

2.2 数据划分与数据增强

为避免测试集信息泄漏，本文不把测试集用于任何调参。具体做法是：在 50000 张训练图像中，按类别分层抽取 500 张作为验证集，剩余 45000 张用于训练；模型选择、早停和学习率策略均只看验证集；测试集只在训练结束后评估一次。这样得到的测试数字更接近“未见数据”的表现。

训练增强由三部分组成：先对图像四周零填充 4 像素，再随机裁剪出 32×32 区域，随后以 0.5 的概率水平翻转；最后施加 Cutout，将 8×8 的随机方形区域置零。前两者对应平移与镜像不变性，Cutout 则迫使网络不要过度依赖单个局部区域。验证与测试阶段不做随机增强，只做中心区域（完整图像）的归一化。

归一化使用 CIFAR-10 的全局均值 (0.4914, 0.4822, 0.4465) 与标准差 (0.2470, 0.2435, 0.2616)。输入张量从 NHWC 转为 PyTorch 的 NCHW，并使用 channels-last 内存格式，以改善 CPU 卷积的访存效率。

2.3 评价指标

分类质量使用五个指标：**Accuracy** 为 Top-1 准确率；**Top-5 Accuracy** 衡量正确类别是否进入前五；**Macro-F1** 对 10 个类别等权平均，能反映类别不均衡时的整体表现；**NLL** 为负对数似然，关注概率质量是否放在正确类别上；**ECE**（Expected Calibration Error，15 个置信度分箱）衡量置信度与真实正确率之间的偏差。ECE 越低，说明模型“知道自己有多确定”。

部署成本使用四个指标：**参数量**（可训练参数个数）、**MACs**（每张图像一次前向的乘加次数，FLOPs = 2 × MACs）、**模型文件大小**（float32 参数占用）与 **CPU 推理延迟**。延迟在 batch=1 和 batch=128 两种设置下测量，前者对应单张在线推理，后者对应吞吐。所有延迟测量均在 `torch.inference_mode()` 下进行，先 warmup 再取 20 次的中位数与 P90，以减少缓存和调度噪声。

需要区分“理论计算量”和“实际延迟”：MACs 只统计乘加次数，不包含内存访问、逐通道卷积的循环开销、算子融合与线程调度。因此，MACs 更低的模型不一定在 CPU 上更快。本文把两者同时报告，正是为了观察这一差异。

2.4 实验环境

实验在 Windows 11 的 CPU 环境中完成，未使用 GPU。Python 3.12.7，PyTorch 2.12.0+cpu，torchvision 0.27.0+cpu，NumPy 2.2.6；CPU 为 Intel Core Ultra 9 185H（16 核 22 线程，支持 AVX2，不支持 AVX-512 BF16），内存 32 GB。训练时固定 16 个

PyTorch 线程，并把 Windows 电源计划设为“高性能”，以减少降频带来的 epoch 时间波动。

在该环境下，ResNet-20 一个 epoch（45000 张训练图 + 5000 张验证图）约需 65–75 秒，具体受后台负载和 CPU 温度影响；六个模型各 60 epoch 的顺序训练耗时数小时。报告中的训练时间取自每个模型 history JSON 的逐 epoch 计时，而不是估算值；唯一的例外是 GELU 训练期间的一次系统挂起，表中用中位数替换了异常 epoch。

第 3 章　模型设计

3.1 ResNet-20 基线

CIFAR 风格 ResNet-20 由 He 等人在残差学习工作中提出：一个 3×3 卷积作为 stem，随后三个 stage，每个 stage 包含 3 个 BasicBlock，通道数分别为 16、32、64；后两个 stage 的第一个 block 使用 stride=2 下采样，其余 block 保持分辨率。最后经全局平均池化和一个 10 类全连接层输出 logits。总参数量约 272,474，单张图像约 40.81M MACs。

每个 BasicBlock 的主分支为“3×3 卷积 → BatchNorm → ReLU → 3×3 卷积 → BatchNorm”，再加一条恒等短路（当通道数或分辨率变化时用 1×1 卷积 + BN 调整），相加后再经过 ReLU。残差连接使梯度可以沿短路直接回传，是训练更深网络的关键。

$$y = \mathrm{ReLU}(\, F(x, \{W_i\}) + x \,)$$

式 3-1 BasicBlock 的残差形式（恒等短路时）

3.2 深度可分离卷积

标准 3×3 卷积在 C_in 个输入通道和 C_out 个输出通道之间做全连接，参数量为 C_in × C_out × 3 × 3。深度可分离卷积把它分解为两步：先对每个输入通道独立做 3×3 卷积（depthwise，groups=C_in），再用 1×1 卷积做通道混合（pointwise）。参数量变为：

$$\mathrm{C_in} \times 3 \times 3 \; + \; \mathrm{C_in} \times \mathrm{C_out}$$

式 3-2 深度可分离卷积的参数量

两者之比约为 1/C_out + 1/9。当 C_out=16 时，理论参数量约为标准卷积的 17.4%，即减少约 82.6%；当通道数增大时，比例进一步接近 1/9。MACs 也有类似的下降。直观上，深度可分离卷积把“空间滤波”和“通道混合”解耦，用更少的乘加完成同样的感受野。

但 depthwise 卷积的每个输出通道只依赖一个输入通道，计算密度低、访存模式不连续；在 CPU 上，它往往无法像标准卷积那样充分利用向量化和多线程，因此实际延迟可能不降反升。这一点将在第 4 章用实测数据检验。

3.3 SE 通道注意力

Squeeze-and-Excitation（SE）模块先对每个通道做全局平均池化，得到一个 C 维向量 z；再用两层 1×1 卷积（中间维度为 max(4, C/8)）学习通道之间的非线性关系，最后用 sigmoid 得到 0 到 1 的通道权重 s，并与原特征图逐通道相乘：

$$z = \mathrm{AvgPool}(x),\ s = \sigma(W2 \cdot \mathrm{ReLU}(W1 \cdot z)),\ x' = s \odot x$$

式 3-3 SE 模块的通道重标定

本文把 SE 放在每个 BasicBlock 的第二个 BN 之后、残差相加之前，只重标定残差分支，不改变短路本身。这样既能让注意力调制非线性特征，又保留了恒等映射的梯度通路。设 h=max(4, C/8)，两层 1×1 卷积均带偏置，则 SE 的额外参数量为 2C·h + h + C；当 C=64 时 h=8，共 1,096 个参数。相对整体很小，但引入了一次全局池化和两次 1×1 卷积，会增加少量延迟。

3.4 GELU 激活

GELU（Gaussian Error Linear Unit）用标准正态分布的累积分布函数对输入加权：

$$\mathrm{GELU}(x) = 0.5 \cdot x \cdot (1 + \mathrm{erf}(x / \sqrt{2}))$$

式 3-4 GELU 的定义

与 ReLU 的硬截断相比，GELU 在负半轴被平滑地压制而不是直接置零，在 0 附近也更平滑。它在 Transformer 中被广泛使用，也被认为可能改善深层网络的优化。但 GELU 需要计算 erf 或 tanh 近似，在 CPU 上通常比 ReLU 更慢；其收益是否足以抵消这一开销，需要实验判断。本文使用 `nn.GELU(approximate='none')`，即精确版本。

3.5 GroupNorm 归一化

BatchNorm 在训练时对一个 mini-batch 内的样本统计均值和方差，因此依赖 batch size，并在训练/推理之间存在行为差异。GroupNorm 把通道分成 G 组，在每组内部跨

通道和空间位置统计，不依赖 batch 维度。本文取 G=8；当通道数为 16、32、64 时都能整除。GroupNorm 为每个通道保留一对 γ、β，参数量与 BatchNorm 相同，但每个样本的统计更稳定，适合小 batch 场景。

在 CPU 上，GroupNorm 需要对每个样本、每个组分别做归约，访存模式不如 BatchNorm 的逐通道归约规则，因此可能更慢。本文把 GroupNorm 作为“归一化方式”的对照，观察其在 batch=256 的训练和 batch=1 的推理下分别带来什么影响。

3.6 六个模型变体

六个模型只在上述组件上不同，深度、宽度、stage 结构、分类头和初始化完全一致。表 3-1 给出配置与理论成本，图 3-1 对比了基线 block 与 DS+SE block 的数据流。

模型	归一化	激活	深度可分离	SE	参数量	MACs (M)
R20	BN	RELU	否	否	272,474	40.81
R20-DS	BN	RELU	是	否	40,522	6.65
R20-SE	BN	RELU	否	是	277,082	40.82
R20-GELU	BN	GELU	否	否	272,474	40.81
R20-GN	GN(8)	RELU	否	否	272,474	40.81
R20-DS-SE	BN	RELU	是	是	45,130	6.66

表 3-1 六个模型的配置与理论成本

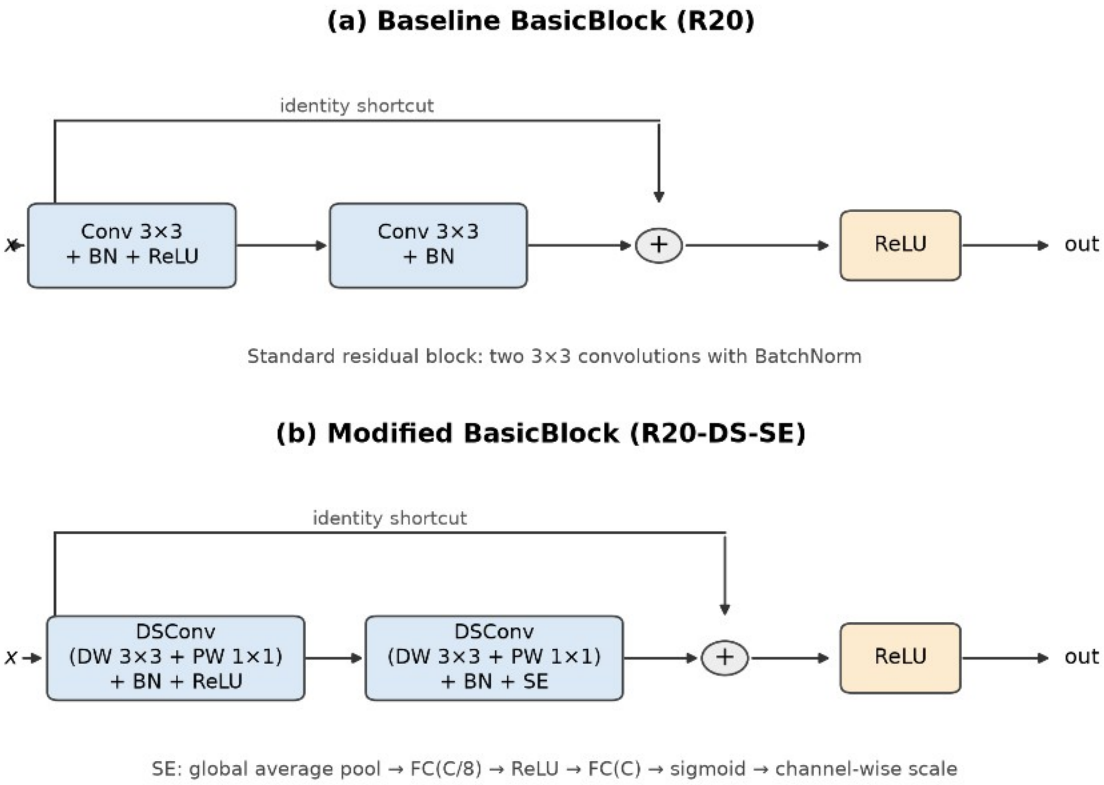


图 3-1 基线 BasicBlock 与 DS+SE BasicBlock 的结构对比

3.7 本章小结

本章从残差块出发，把 CNN 的结构设计拆成“空间卷积、通道注意力、激活函数、归一化方式”四个可替换组件。深度可分离卷积把标准 3×3 卷积分解为逐通道空间卷积与 1×1 通道混合，理论上把乘加量从 C_in×C_out×9 降到 C_in×9+C_in×C_out；SE 用全局池化和两层瓶颈全连接学习通道权重；GELU 用平滑的高斯门控替代 ReLU 的硬截断；GroupNorm 则把归一化从 batch 维度移到通道组内。

六个模型共享同一深度、宽度、stage 划分、分类头与初始化，只在上表所列组件上不同。这样，第 4 章观察到的精度与成本差异才能归因于结构本身，而不是训练轮数、数据划分或超参数的偶然变化。

第 4 章　训练结果与结构分析

4.1 统一训练配方

为了让结构差异成为唯一变量，六个模型使用完全相同的训练配方。数据为 45000 张训练图、5000 张验证图；batch size 256；SGD + momentum 0.9 + Nesterov；基础学习率 0.1，5 个 epoch 线性 warmup，随后余弦衰减到 1e-4；weight decay 5e-4；label smoothing 0.1；Cutout 8；共 60 个 epoch；随机种子 42。

超参数	取值	超参数	取值
Epochs	60	Batch size	256
Optimizer	SGD + Nesterov	Momentum	0.9
Base LR	0.1	Min LR	1e-4
Warmup	5 epochs	Schedule	cosine
Weight decay	5e-4	Label smoothing	0.1
Cutout	8	Seed	42
输入归一化	CIFAR-10 mean/std	推理精度	float32 CPU

表 4-1 六个模型共享的训练配方

每个模型训练完成后，按验证集准确率选择最佳 checkpoint，再在 10000 张测试图上评估一次。基线的最佳 epoch 为 56，验证准确率 89.00%，测试准确率 88.54%；六个模型的总训练时间见 4.4 节。需要说明的是，ResNet-20 原论文使用 200 epoch 训练并在 CIFAR-10 上报告 91.25%；本文将统一预算设为 60 epoch，是为了在纯 CPU 上完成六个模型的全部训练。因此表中的绝对精度低于原论文的长训练结果，但六个模型共享同一数据划分、增强与优化预算，结构之间的相对差异仍然可比。

4.2 训练曲线

图 4-1 给出六个模型的训练损失与验证准确率。所有模型都经历了相似的三个阶段：warmup 阶段损失快速下降，中间阶段在数据增强下波动，最后在余弦学习率接近 1e-4 时收敛。曲线之间的差异主要出现在中后段，说明结构改动影响的是特征学习效率，而不是单纯的初始化差异。

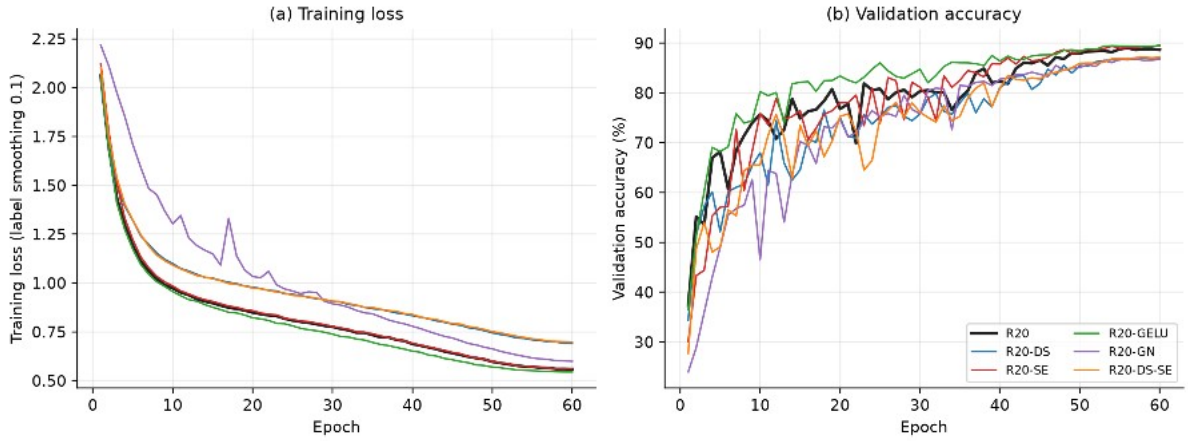


图 4-1 六个模型的训练损失与验证准确率

4.3 总体结果

表 4-2 汇总了测试集上的分类指标，表 4-3 汇总了部署成本。表中的“ΔAcc”均相对基线 R20 计算，正数表示优于基线。

模型	参数量(M)	MACs(M)	Test Acc	ΔAcc(pp)	Top-5	Macro-F1	ECE
R20	0.272	40.81	88.54%	+0.00	99.28%	0.8847	0.0396
R20-DS	0.041	6.65	86.68%	-1.86	99.41%	0.8667	0.0407
R20-SE	0.277	40.82	89.07%	+0.53	99.26%	0.8903	0.0485
R20-GELU	0.272	40.81	89.34%	+0.80	99.32%	0.8929	0.0424
R20-GN	0.272	40.81	85.76%	-2.78	98.86%	0.8569	0.0506
R20-DS-SE	0.045	6.66	86.45%	-2.09	99.37%	0.8642	0.0545

表 4-2 六个模型的测试集分类指标（ΔAcc 相对基线）

模型	模型大小(MB)	batch=1 延迟(ms)	batch=128 吞吐(img/s)	有效训练时间(min)
R20	1.09	1.95	4049	64.8
R20-DS	0.16	3.08	5980	40.2
R20-SE	1.11	2.96	3338	60.2
R20-GELU	1.09	2.37	3335	55.9
R20-GN	1.09	2.19	2926	61.5
R20-DS-SE	0.18	3.88	4960	46.0

表 4-3 模型大小、CPU 推理延迟与训练时间

延迟在 16 线程、channels_last、torch.inference_mode 下测量；六个模型交错进行 5 轮，batch=1 每模型取 50 次、batch=128 每模型取 25 次，表中为中位数。交错测量可避免“先测的模型承担冷缓存与升频开销”造成的系统偏差。

表 4-3 的训练时间按逐 epoch 计时求和。GELU 训练期间发生过一次 Windows 系统挂起（单个 epoch 的墙钟时间被拉长到 1168.8 秒），若直接使用墙钟总时长会把挂起时间误算成模型开销；因此表中把该异常 epoch 替换为中位数后再求和，其余模型仍为实际计时。

从表 4-2 可以看到，测试准确率最高的是 **R20-GELU（89.34%）**，最低的是 R20-GN（85.76%）。六个模型的 Top-5 准确率都明显高于 Top-1，说明错误主要集中在少数容易混淆的类别之间。ECE 的差异则反映了不同结构对置信度校准的影响。

4.4 精度—成本权衡

图 4-2 把测试准确率分别对参数量、MACs 和 batch=1 延迟作图。左、中两图的横轴是对数坐标。如果只看前两张图，深度可分离卷积看起来是“免费午餐”：参数和 MACs 都下降了一个数量级，而精度只变化了几个点。但右图把延迟纳入后，结论就复杂得多。

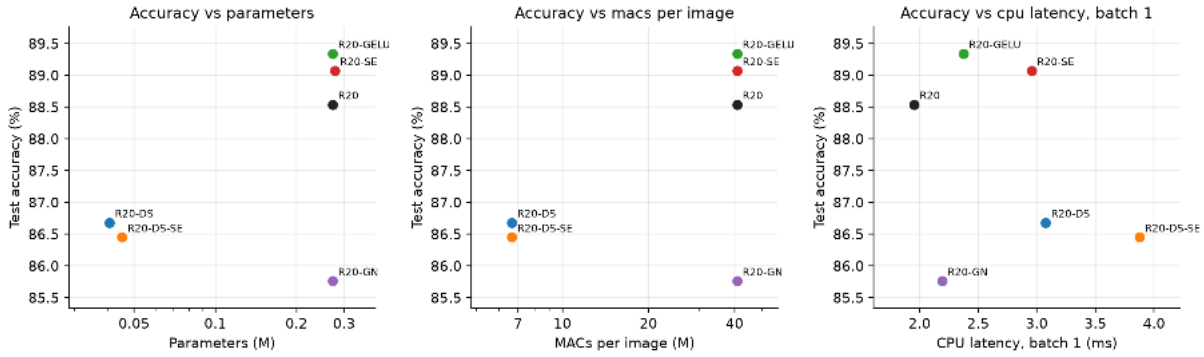


图 4-2 测试准确率与参数量、MACs、CPU 延迟的关系

深度可分离卷积（R20-DS）把参数量从 272,474 降到 40,522，降幅 85.1%；MACs 从 40.81M 降到 6.65M，降幅 83.7%。测试准确率从 88.54% 变为 86.68%（-1.86 个百分点）。然而 batch=1 延迟从 1.95 ms 变为 3.08 ms，即 **+57.4%**。

这一结果说明，在 CPU 上 depthwise 卷积的低计算密度并没有转化为低延迟。原因主要有三：其一，depthwise 卷积的每个通道独立计算，难以利用标准卷积的高度优化内核，向量化程度低；其二，逐通道访存和 1×1 卷积的额外启动开销增加了内存带宽压力；其三，PyTorch 在 CPU 上对 groups 较大的卷积支持不如稠密卷积成熟。因此，“参数少、MACs 少”只说明模型更小，不说明推理更快。对边缘部署而言，必须同时测量目标硬件上的真实延迟。

不过，若把观察量换成 batch=128 的吞吐，R20-DS 为 5980 img/s，反而高于基线的 4049 img/s（+47.7%）。这与 batch=1 的结论并不矛盾：单张延迟由算子启动、调

度和访存开销主导，大 batch 下 depthwise 的低 MACs 更容易摊薄固定开销。因此部署评价必须区分“单请求延迟”和“批量吞吐”，不能只报一个。

SE（R20-SE）只增加 4,608 个参数（约 1.7%），MACs 几乎不变（40.82M vs 40.81M）。测试准确率为 89.07%，相对基线 +0.53 个百分点；batch=1 延迟为 2.96 ms，相对基线 +51.3%。SE 的增益为正，说明在本任务上“哪些通道更重要”确实是有效信息，瓶颈结构以很小的参数代价提供了图像级的通道上下文。

GELU（R20-GELU）不改变参数量与 MACs，测试准确率为 89.34%，相对基线 +0.80 个百分点；batch=1 延迟为 2.37 ms，相对基线 +21.5%。在本例中 GELU 略高于基线，说明平滑激活对优化有一点帮助；但 GELU 的 erf 计算通常比 ReLU 更昂贵，收益是否值得取决于精度预算与延迟约束。

GroupNorm（R20-GN）的参数量与基线相同，测试准确率为 85.76%，相对基线 -2.78 个百分点；batch=1 延迟为 2.19 ms，相对基线 +12.2%。GroupNorm 不依赖 batch 统计，在小 batch 或分布偏移场景下通常更稳；但在 batch=256 的训练设置下，BatchNorm 的跨样本统计本身就有正则化效果，GroupNorm 未必占优。其逐样本、逐组的归约也可能增加 CPU 开销，但本次实测延迟受内核实现与线程数影响，未必与理论开销同向。

组合模型 R20-DS-SE 的参数量为 45,130，MACs 为 6.66M，测试准确率 86.45%，相对 R20-DS -0.23 个百分点，相对基线 -2.09 个百分点。其 batch=1 延迟为 3.88 ms，相对基线 +98.4%。

相对 R20-DS，SE 的准确率变化为 -0.23 个百分点，基本持平；在本训练预算下，注意力带来的收益被额外的门控开销抵消。因此，“轻量卷积 + 注意力”是否值得，取决于部署端更在意模型体积、吞吐还是单张延迟；不能仅凭参数量更小就推断精度更高或更快。

4.5 逐类精度与混淆矩阵

图 4-3 对比了基线与 R20-DS-SE 的逐类召回率，图 4-4 给出两者的行归一化混淆矩阵。CIFAR-10 中猫/狗、鹿/马、飞机/船等类别在低分辨率下外观接近，是主要的混

淆来源。若某个结构在整体准确率上只差零点几个点，但在“猫/狗”等关键类别上差异明显，则不能简单地说两者等价。

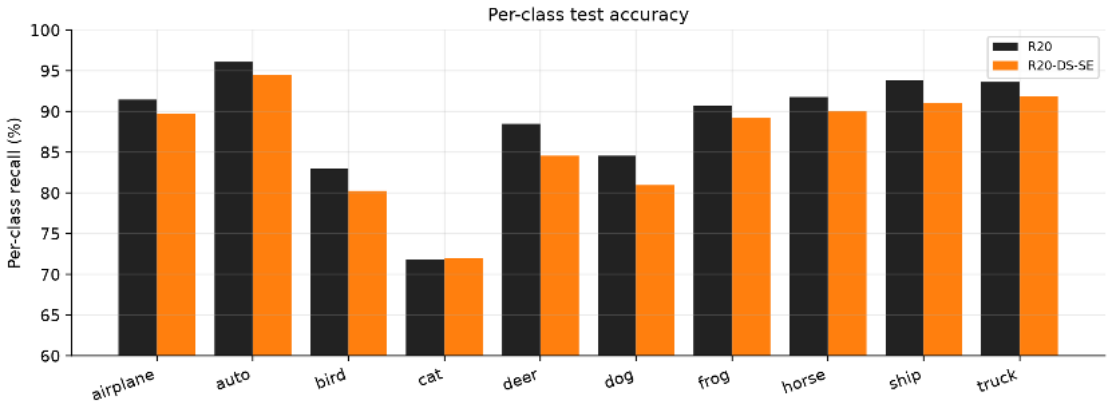


图 4-3 基线 R20 与 R20-DS-SE 的逐类测试召回率

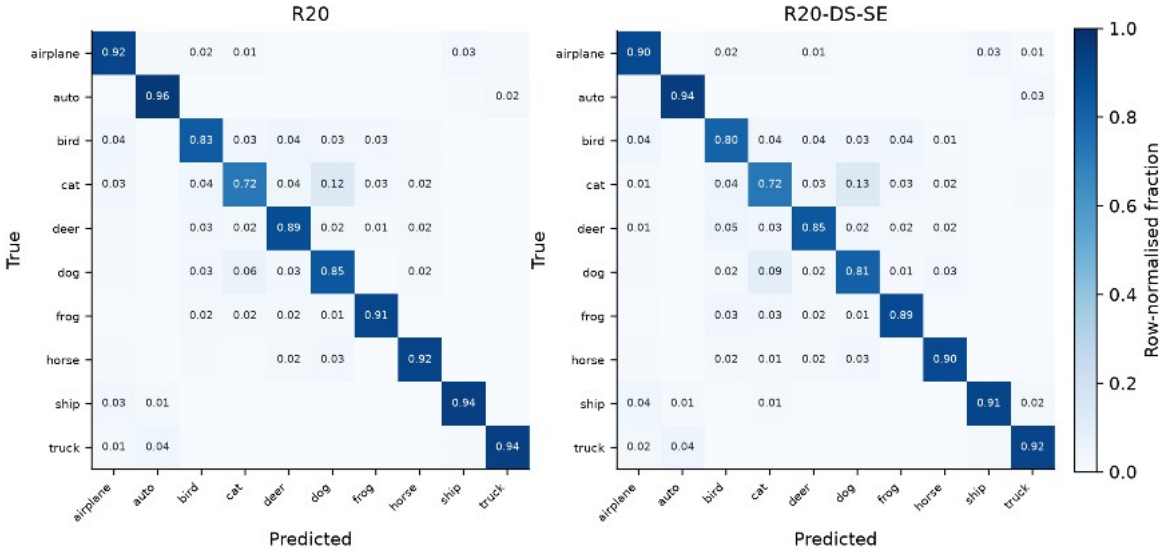


图 4-4 R20 与 R20-DS-SE 的测试集混淆矩阵（行归一化）

从逐类结果看，R20-DS-SE 相对基线下降最多的是 **鹿**（-3.9 个百分点），提升最多的是 **猫**（+0.2 个百分点）。这说明结构改动并不是在所有类别上均匀生效；参数量更小的模型可能对纹理简单、形状明确的类别更友好，而对细粒度差异较大的类别更吃力。

4.6 小结

本章的受控实验给出三点结论。第一，深度可分离卷积在参数量与 MACs 上优势巨大，但在 CPU 上不一定更快，部署前必须实测延迟。第二，SE 以很小的参数代价提供了通道重标定能力，本次相对基线为 +0.53 个百分点；GELU 与 GroupNorm 相对基线的变化分别为 +0.80 个百分点 和 -2.78 个百分点。这些差异都应在“单次 60 epoch 训练”的误差范围内谨慎解读，不能直接外推为普遍结论。第三，组合模型把参数量压到 45,130，但其测试准确率相对 R20-DS 的变化为 -0.23 个百分点；“小体积”与“高精度”并不自动同时成立，需要按部署目标取舍。

第 5 章　低能见度鲁棒性与场景应用

5.1 场景动机与退化设计

道路场景中的低能见度主要由雾、夜间低照度、相机抖动、传感器噪声和图像压缩造成。真实数据的采集需要对应天气与道路条件，标注成本高；在课程作业中，更可控的做法是在 CIFAR-10 测试集上施加合成退化，固定退化参数与随机种子，比较不同结构在相同扰动下的相对表现。

本文实现五类退化，每类 5 个强度（severity 1–5），共 25 个测试条件。所有退化都在 uint8 NHWC 图像上进行，再统一归一化；同一强度下所有模型使用相同的随机种子，因此比较是配对的。

退化	实现方式	severity 1 → 5 的关键参数
雾	与亮灰蓝色 [232,236,242] 按透明度 α 混合	$\alpha \approx 0.20 \rightarrow 0.63$
低照度	Gamma 压暗 + 高斯读出噪声	$\gamma = 1.24 \rightarrow 2.20$，$\sigma = 3.5 \rightarrow 11.5$
运动模糊	0/45/90/135° 线性运动核，逐图随机方向	核长 $3 \rightarrow 11$
高斯噪声	加性高斯噪声	$\sigma = 6 \rightarrow 30$
JPEG 压缩	PIL JPEG 编码再解码	quality $52 \rightarrow 12$

表 5-1 五类合成退化的实现

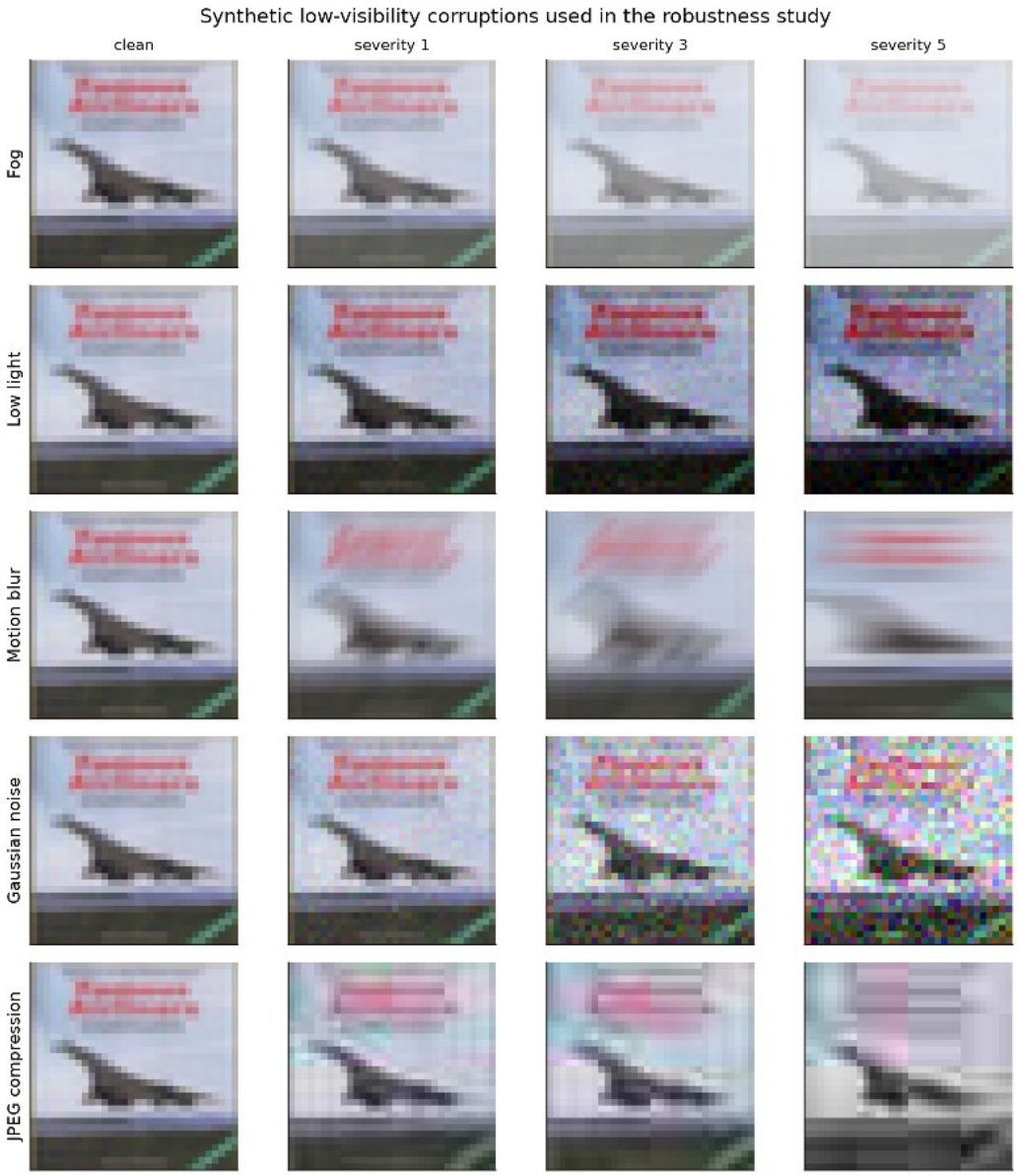


图 5-1 同一张测试图在干净、severity 1/3/5 下的退化示例

这些退化只模拟了低能见度的一部分统计特征，并不等价于真实雾天成像：真实雾的浓度随深度变化，夜间还有光源眩光与色偏，运动模糊与曝光时间耦合。因此，本章的结论应理解为“在受控合成退化下的相对鲁棒性”，而不是对真实道路场景的绝对性能承诺。

5.2 全模型鲁棒性结果

表 5-2 列出每个模型在干净测试集、五类退化（对 severity 1–5 取平均）以及全部 25 个条件上的平均与最差准确率。图 5-2 给出准确率随 severity 变化的曲线，图 5-3 用热力图展示模型与退化类型的交互。

模型	Clean	雾	低照度	运动模糊	噪声	JPEG	平均	最差
R20	88.54%	78.35%	71.97%	33.87%	42.88%	60.33%	57.48%	15.40%
R20-DS	86.68%	73.64%	66.35%	34.59%	36.05%	59.05%	53.93%	13.05%
R20-SE	89.07%	76.81%	73.92%	34.56%	50.70%	60.63%	59.32%	17.03%
R20-GELU	89.34%	78.79%	72.59%	36.61%	44.85%	63.08%	59.19%	17.79%
R20-GN	85.76%	83.21%	73.26%	50.02%	58.17%	63.47%	65.63%	29.84%
R20-DS-SE	86.45%	71.36%	65.35%	34.29%	39.09%	60.45%	54.11%	14.06%

表 5-2 六个模型在干净与退化测试集上的准确率（退化列为 severity 1–5 平均）

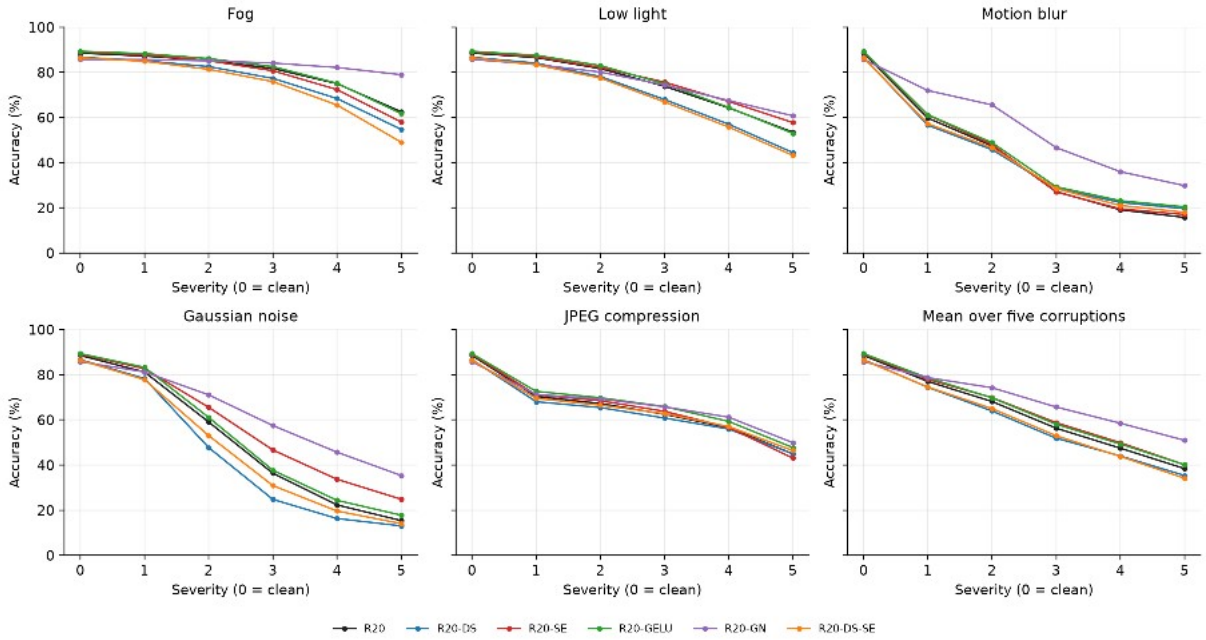


图 5-2 准确率随退化强度变化的曲线（横轴 0 为干净图像）

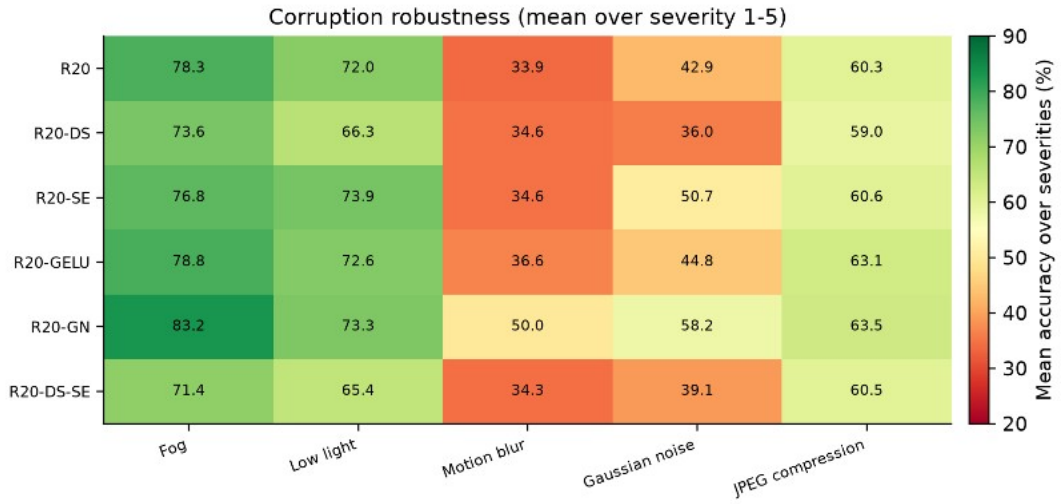


图 5-3 模型 × 退化类型的平均准确率热力图

基线在 25 个退化条件上的平均准确率为 57.48%，相对干净图像下降 31.06%。鲁棒性最好的模型是 **R20-GN（65.63%）**，最差的是 **R20-DS（53.93%）**。从退化类

型看，雾（所有模型平均 77.03%）、低照度（所有模型平均 70.57%）、运动模糊（所有模型平均 37.32%）、高斯噪声（所有模型平均 45.29%）、JPEG 压缩（所有模型平均 61.17%）。

一个值得注意的现象是：干净精度最高的 R20-GELU 并不是退化条件下平均准确率最高的模型，后者是 R20-GN。这说明仅凭干净测试集选型，可能会低估模型在低能见度下的相对表现。鲁棒性比较应同时看“绝对退化精度”和“相对干净精度的下降幅度”：前者更贴近部署体验，后者更能反映结构对分布偏移的敏感度。

分退化类型看，R20-GN 的优势主要集中在运动模糊（50.02%，基线 33.87%）和高斯噪声（58.17%，基线 42.88%）上；在雾和低照度上，它与基线的差距要小得多。一种解释是：GroupNorm 在推理时对每个样本独立统计，局部像素被模糊或加噪后，归一化会重新缩放该样本的特征，而 BatchNorm 使用训练集累积的 running statistics，对这类局部扰动更敏感。但这一解释仍是假设，需要消融归一化的统计方式才能确认；同时 R20-GN 的干净精度最低，说明鲁棒性提升不是“免费”的。

5.3 退化数据微调

为了验证“用退化数据继续训练能否提升低能见度表现”，本文从 R20-DS-SE 的最佳干净 checkpoint 出发，再做 15 个 epoch 的微调：每个 batch 以 0.5 的概率施加随机类型、随机强度的退化，其余设置与主训练一致但学习率降为 0.01。模型选择使用一个混合验证集：5000 张干净验证图 + 每类退化各 1000 张（severity=3），共 10000 张。这样做的目的是避免只优化干净精度或只优化退化精度。

条件	微调前	微调后	变化
干净测试集	86.45%	85.65%	-0.80 个百分点
25 个退化条件平均	54.11%	65.70%	+11.59 个百分点
最差退化条件	14.06%	24.51%	+10.45 个百分点

表 5-3 R20-DS-SE 退化微调前后的准确率

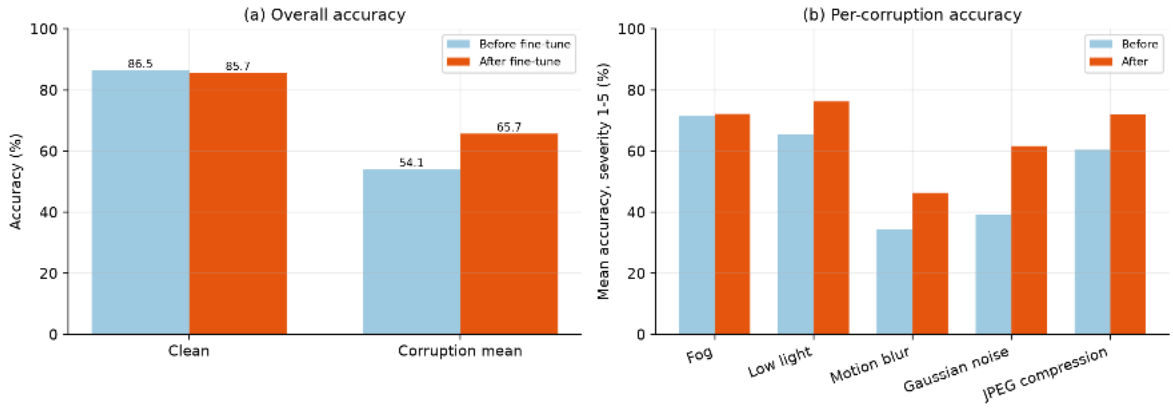


图 5-4 退化微调前后的整体与分类型准确率

微调后，退化平均准确率变化 **+11.59 个百分点**，最差条件变化 +10.45 个百分点；干净准确率变化 -0.80 个百分点。这体现了典型的“鲁棒性—干净精度”权衡：退化增强让模型见过更多低质量输入，从而在退化条件下更稳，但牺牲了一部分干净数据的判别边界。如果应用场景明确以低能见度为主，这种交换是合理的；如果干净与退化图像混合出现，则应把干净验证集纳入模型选择，或采用更小的微调学习率与更少的 epoch。

5.4 Grad-CAM 可解释性

Grad-CAM 通过目标类别对最后一个卷积 stage 特征的梯度，得到一张 32×32 的重要性热力图。本文对基线 R20 与 R20-DS-SE 分别输入干净图像和 severity=4 的雾天图像，比较两者关注区域的变化。热力图越接近红色，表示该位置对预测的贡献越大。

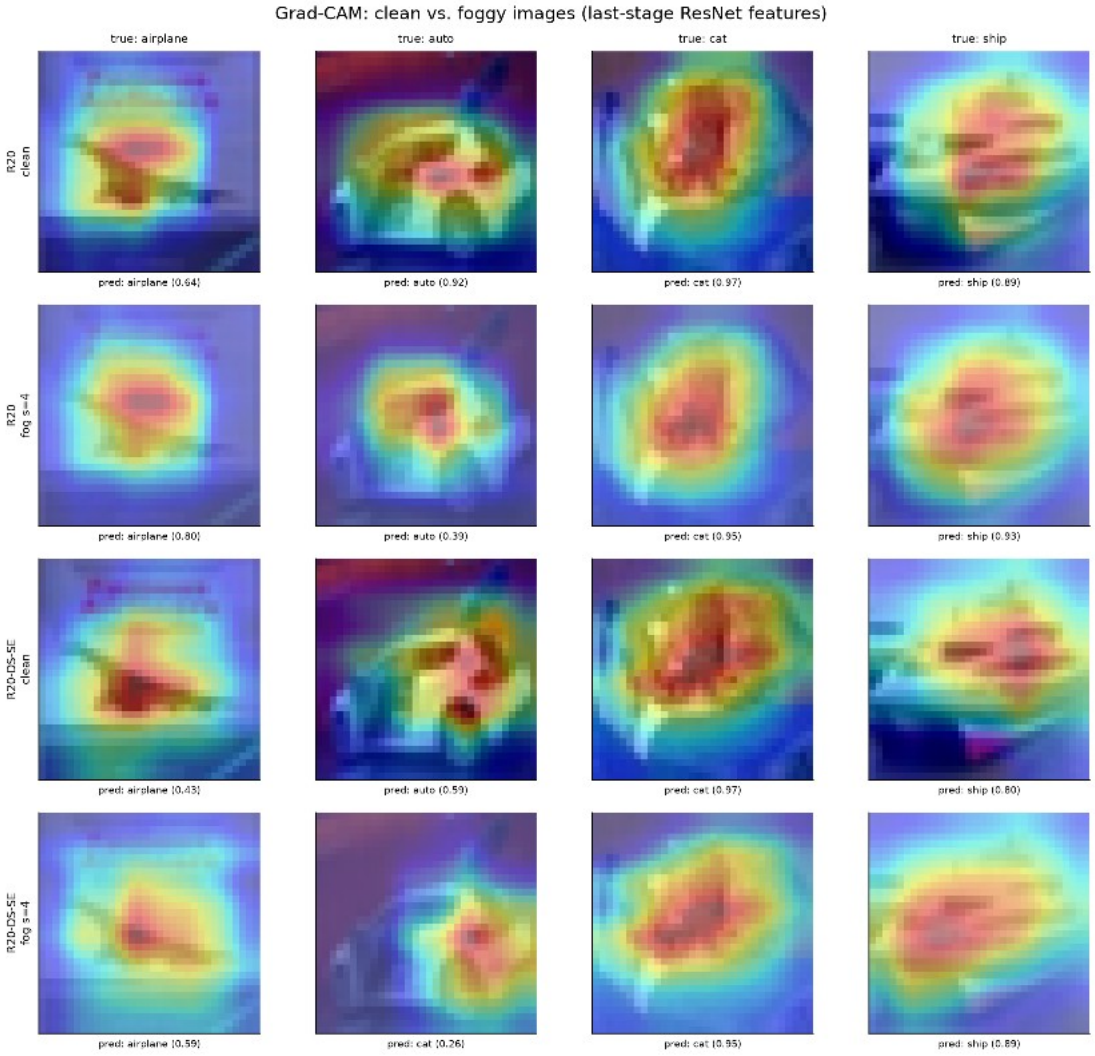


图 5-5 R20 与 R20-DS-SE 在干净/雾天图像上的 Grad-CAM

在干净图像上，两个模型的高响应区域通常集中在目标主体（如飞机机身、船的轮廓、猫的头部）；在雾天图像上，响应会变得更分散，部分样本转向背景或边缘。若某个模型在雾天仍能把响应集中在主体上，说明其特征对对比度下降更不敏感；反之，若响应大面积转移到天空或地面，则 accuracy 的下降就有了可视化的解释。Grad-CAM 是定性证据，不能替代定量指标，但可以帮助检查模型是否“看对了地方”。

5.5 交互演示系统

为了把训练好的模型用于实际交互，本文实现了一个 Flask 演示：用户上传图片，选择模型（默认 R20-DS-SE）、退化类型与强度，服务端完成中心裁剪、缩放到 32×32、归一化、前向推理与 Grad-CAM，返回 Top-3 类别、置信度、网络输入和热力图叠加结果。演示还提供 `/health` 接口用于健康检查，以及 `?demo=1` 示例自动运

行：页面载入内置测试图并自动完成一次指定退化的推理，便于在无外部图片时复现截图。



图 5-6 Flask 演示界面（示例自动运行：R20 对雾天样本的 Top-3 与 Grad-CAM）

演示的意义在于验证“模型—预处理—推理—可视化”整条链路可运行，而不是提供一个可直接用于真实道路的产品。CIFAR-10 的 32×32 输入与真实车载相机的分辨率差距很大；若要把系统扩展到真实场景，需要先做目标检测或感兴趣区域裁剪，再在域内数据上重新训练，并补充雨、雪、夜间眩光等退化类型。

5.6 场景适配的局限

本章实验存在三方面局限。第一，合成退化只覆盖了低能见度的一部分统计特征，真实雾天的深度相关散射、夜间色偏与运动模糊的点扩散函数都更复杂。第二，CIFAR-10 的类别是通用物体，不是道路参与者；“低能见度道路场景”在这里是方法验证的场景隐喻，而不是端到端的道路感知系统。第三，鲁棒性评估只用了准确率，没有评估检测框、分割掩码或时间一致性。因此，报告中的结论应定位为“结构在受控退化下的相对行为”，而不是实际部署的性能保证。

第 6 章　结论与展望

6.1 结论

本文在 CIFAR-10 上完成了 ResNet-20 及其五个结构变体的受控训练与评估。基线测试准确率为 88.54%；深度可分离卷积把参数量降至 40,522、MACs 降至 6.65M，但单张 CPU 延迟由 1.95 ms 变为 3.08 ms（+57.4%），并未随 MACs 同比下降；这提醒我们，模型压缩的收益必须在目标硬件上测量，不能只用 FLOPs 代替延迟。

在结构组件层面，SE、GELU、GroupNorm 相对基线的测试准确率变化分别为 +0.53 个百分点、+0.80 个百分点、-2.78 个百分点。SE 用 4,608 个额外参数带来了正提升，但幅度不及 GELU。其中 GELU 相对基线为 +0.80 个百分点，GroupNorm 为 -2.78 个百分点；单次训练预算下，这一差异不足以证明 GELU 普遍优于 ReLU。

组合模型 R20-DS-SE 的参数量为 45,130，测试准确率 86.45%，相对 R20-DS 的变化为 -0.23 个百分点。它适合作为“体积优先”场景的候选，但是否部署仍取决于延迟预算；参数少并不自动意味着精度更高或更快。

在低能见度模拟中，所有模型的准确率都随退化强度下降；对 R20-DS-SE 做退化微调后，25 个退化条件的平均准确率由 54.11% 变为 65.70%，干净准确率由 86.45% 变为 85.65%。这说明退化增强是一种有效但需要权衡的域适应手段。

本作业的工程价值在于“可复现的闭环”：数据准备、训练、评估、成本测量、鲁棒性、微调、可视化与演示均由脚本完成，报告数字由 JSON 自动生成。相比单独追求一个高准确率，这种闭环更能支撑对 CNN 组件的理解，也更接近真实项目的工作方式。

6.2 不足与改进方向

- **训练预算有限**：60 epoch 的短训练可能低估了 SE、GELU 等模块的潜力；更长的训练与多次随机种子重复可以给出置信区间。
- **延迟测量范围有限**：只在一种 CPU、一个 PyTorch 版本上测量；结论应扩展到 ARM、GPU、ONNX Runtime 与量化推理。

- **退化类型有限**：可加入雨、雪、夜间眩光、色偏与混合退化，并采用 CIFAR-10-C 等标准基准做对照。
- **场景不是真实道路数据**：未来可在 BDD100K、Cityscapes 等数据上做低能见度子集评估，或先检测再分类。
- **可解释性仍是定性**：可加入 Grad-CAM 的定量指标（如与标注框的 IoU）与失败案例分析。

后续工作可以从两个方向展开：一是把结构搜索与硬件感知延迟结合，在目标设备上做真实的 NAS/剪枝；二是把退化鲁棒性从分类扩展到检测与分割，研究低能见度下的多任务一致性。

参考文献

[1] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition. CVPR, 2016.

[2] He K., Zhang X., Ren S., Sun J. Identity Mappings in Deep Residual Networks. ECCV, 2016.

[3] Howard A. G., Zhu M., Chen B., et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861, 2017.

[4] Hu J., Shen L., Sun G. Squeeze-and-Excitation Networks. CVPR, 2018.

[5] Hendrycks D., Gimpel K. Gaussian Error Linear Units (GELUs). arXiv:1606.08415, 2016.

[6] Wu Y., He K. Group Normalization. ECCV, 2018.

[7] Selvaraju R. R., Cogswell M., Das A., et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV, 2017.

[8] Hendrycks D., Dietterich T. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. ICLR, 2019.

[9] Krizhevsky A. Learning Multiple Layers of Features from Tiny Images. Technical Report, University of Toronto, 2009.

[10] Ioffe S., Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. ICML, 2015.

附录 A 代码结构

仓库按“数据—模型—训练—评估—应用”分层。核心文件如下：

路径	作用
src/models.py	ResNet-20、DS 卷积、SE、GELU、GroupNorm 与六个模型注册
src/data.py	npz 读取、分层划分、增强、归一化、批迭代
src/engine.py	单 epoch 训练、评估、余弦学习率
src/metrics.py	Accuracy、Top-5、NLL、Macro-F1、ECE、混淆矩阵
src/flops.py	参数量与 MACs 统计
src/corruptions.py	五类合成退化
src/robustness.py	退化测试套件
src/gradcam.py	Grad-CAM
scripts/train.py	单模型训练与 checkpoint
scripts/benchmark.py	成本与延迟测量
scripts/robustness.py	全模型退化评估
scripts/finetune_robust.py	退化数据微调
scripts/visualize.py	报告插图
demo/app.py	Flask 演示

表 A-1 代码结构

附录 B 复现命令

以下命令均在仓库根目录执行，Windows PowerShell 为例：

```
python -m pip install -r requirements.txt

python -m scripts.prepare_data --source torchvision --output-dir data

python -m scripts.train --model resnet20 --epochs 60 --cutout 8 --threads 16

python -m scripts.evaluate --model resnet20

python -m scripts.benchmark --threads 16

python -m scripts.robustness --threads 16

python -m scripts.finetune_robust --model resnet20_ds_se --epochs 15 --threads 16

python -m scripts.visualize

python -m scripts.build_report

python demo/app.py --port 5000
```

六个模型可按 README 中的循环依次训练；`--resume` 可从 `checkpoints/<model>_last.pt` 断点续训。所有脚本的随机种子、数据划分和退化种子均已固定，报告中的数字由 `results/metrics/*.json` 直接读取。

附录 C 结果文件说明

文件	内容
results/history/<model>.json	逐 epoch 训练损失、验证准确率、学习率、耗时
results/metrics/<model>_summary.json	参数量、MACs、最佳 epoch、测试指标
results/metrics/<model>_test_logits.npz	测试集 logits 与标签
results/metrics/benchmark.json	batch=1/128 延迟、吞吐、模型大小
results/metrics/robustness.json	六模型 × 五退化 × 五强度的准确率
results/metrics/robustness_summary.csv	干净/退化平均/最差准确率汇总
results/metrics/resnet20_ds_se_robust_summary.json	退化微调前后对比
report/figures/*.png	报告插图

表 C-1 结果文件